# LMS Migration Quality Auditor

Complete Technical Handbook, Human Judgment Calibration, Master Prompts & CLI Commands

| | |
|---|---|
| **Target System:** New LMS Staging Environment | **Authoritative Source:** Legacy LMS (Source of Truth) |
| **Repository:** github.com/jitendera-code/course-auditor | **Primary Goal:** Eliminate False Positives via Human Judgment |

## 1. Core Architectural & Permanent QA Rules

**Rule 1: Legacy is the 100% Authoritative Source of Truth**
A real defect can ONLY exist when Legacy proves that New is incorrect. Never flag New-only observations without contrasting Legacy baseline proof.

**Rule 2: Human-Readable Student Experience Only**
Do NOT analyze raw HTML markup, CSS class names, DOM hierarchy, or scripts. Compare content strictly as a student reading the screen.

**Rule 3: Ignore Cosmetic Typography & UI Styling**
Bold vs regular words, italics vs normal text, line wrap changes, and speaker/audio icons must NEVER be logged as defects if text is identical.

**Rule 4: Occurrence-Count Duplicate Headings**
Flag a duplicate heading ONLY when New System contains an EXTRA occurrence compared with Legacy.

## 2. Mandatory Defect Reporting Template & Styling

Every defect in Word (.docx), Google Docs, or Markdown must adhere strictly to this case-sensitive structure:

```
[Document Tab / Chapter Heading]

ERROR [N]: [Short Defect Title]
Error Type : [Defect Category]
Location: [Module -> Subtopic -> Slide]
issue: [Clear explanation of why this is a real defect for the student]
Legacy (Expected) : [Exact expected content / presentation from Legacy System]
New System: [Actual deficient content / presentation in the New System]
```

### Color Coding Specifications:

**RED (#B91C1C):** Critical Defects, Dropped Text, Missing Paragraphs, Missing Pages, Broken 404 Links.

**ORANGE (#C2410C):** Duplicate Headings, Table Structure Mismatches, Deformed Callouts / Overlapping Boxes.

**GREEN (#15803D):** Expected / Matching Legacy Content (used for contrast).

## 3. The Complete Master Prompts Collection

Use these exact prompts in Antigravity, ChatGPT, or Claude to enforce human judgment at every stage.

## Prompt 1: Senior Human QA Judge (For Comparing New Slides)

```
You are a Senior LMS Migration QA Auditor. Compare Legacy (Source of Truth) vs New System.
Analyze strictly through the eyes of an enrolled student.

### STRICT IGNORE LIST (NEVER FLAG THESE):
1. TYPOGRAPHY: Do NOT flag bold vs normal text or italics if words are identical.
2. UI & MEDIA ICONS: Do NOT flag missing audio/speaker icons next to headings.
3. HARMFUL PUNCTUATION: Ignore minor punctuation differences (quotes, hyphens).
4. LOCAL ANCHORS: Do NOT flag links containing local export anchors (#_idTextAnchor).
5. COSMETIC TABLES: Do NOT flag shading or border colors if cell data matches.
6. CONTENT SPLIT: If 1 Legacy slide was split across 2 New subslides, it is NOT missing.

### REAL DEFECTS TO CATCH (REPORT ONLY THESE):
- Dropped, truncated, or missing paragraphs, sentences, or bullet points.
- Duplicate headings appearing twice consecutively at top of viewport.
- Altered legal, regulatory, or numeric definitions (e.g. guaranteed vs not guaranteed).
- Missing data rows/columns in tables, or corrupted layout.
- Non-functional / 404 links, or wrong destination URLs.
- Severe visual layout defects (overlapping boxes, broken image floats).

### OUTPUT FORMAT:
If NO real defects, output: 'NO DEFECTS DETECTED (CONTENT VALIDATED)'.
If a defect exists, output strictly in this format:

[Chapter / Topic Name]

ERROR [N]: [Short Title]
Error Type : [Defect Category]
Location: [Topic Path]
issue: [Clear plain-English explanation]
Legacy (Expected) : [Exact expected content]
New System: [Actual deficient content]

[LEGACY CONTENT]: {paste here}
[NEW SYSTEM CONTENT]: {paste here}
```

## Prompt 2: The Report De-Noiser (To Filter 97-Error Cascade Reports)

```
You are an Executive QA Filter. I am pasting an automated error report that contains
too many false positives. Return ONLY genuine, human-relevant defects.

### DISCARD / REMOVE:
- Any error complaining about bold words or italic styling.
- Any error complaining about missing speaker/audio icons.
- Any boilerplate error repeated across 10+ slides.
- Any error complaining about CSS border colors or zebra-striping.
- Any offset-cascade false positive where text simply shifted by 1 slide.

### KEEP ONLY:
- Genuine dropped/truncated text and missing sentences.
- Duplicate back-to-back headings at top of page.
- Missing tables or missing data cells.
- Broken links or altered regulatory meanings.

RAW AUTOMATED REPORT:
{Paste bloated 30-97 error report here}
```

### Prompt 3: Ground-Truth Calibration (Chapters 1–3 Alignment)

```
You are an AI Calibration Expert. I am conducting a Ground-Truth Calibration.
Below are two sets of findings for Chapters 1 to 3:
1. MY MANUAL AUDIT (Human Ground Truth)
2. AUTOMATED MODEL AUDIT

### TASKS:
1. FALSE POSITIVE ANALYSIS: Identify every error automated tool reported that I did NOT.
   Explain WHY (offset shift, bold styling, sentence splitting).
2. FALSE NEGATIVE ANALYSIS: Identify defects I caught that automated tool missed.
3. CALIBRATION RULES: Formulate strict negative rules so the tool never repeats them.

MY MANUAL AUDIT: {Paste your notes for Ch 1-3}
AUTOMATED MODEL AUDIT: {Paste tool output for Ch 1-3}
```

### Prompt 4: Multimodal Visual Layout & Screenshot Inspection

```
You are a Visual Layout QA Auditor. Compare the two screenshots side-by-side:
Screenshot A: Legacy System (Source of Truth)
Screenshot B: New Migrated LMS

### VISUAL AUDIT CRITERIA:
1. CONTAINER INTEGRITY: Look for deformed callouts, unnatural solid colored blocks
   (e.g. solid blue/gray boxes), or missing divider lines.
2. IMAGE FLOATING & OVERLAPPING: Confirm whether diagrams float cleanly beside text or
   if they are broken, misaligned, or overlapping text containers.
3. TITLE DUPLICATION: Check if the viewport header is duplicated twice back-to-back.

Ignore minor pixel shifts or modern font antialiasing. Report only structural defects.
```

### Prompt 5: Corrective Feedback Prompt (When Model Nitpicks)

```
You flagged an error regarding bold/italic font styling in line X.
Reminder: Under our Human-Judgment QA Standard, bold or italic styling variations are
100% HARMLESS and MUST BE IGNORED as modern LMS typography unless actual words were dropped.
Remove that entry and regenerate the cleaned defect list.
```

## 4. Complete CLI & PowerShell Command Reference

All installation, execution, extraction, and GitHub synchronization commands:

### A. Python Package Installation Commands:

```
# Primary installation command (PowerShell / CMD):
python -m pip install requests beautifulsoup4 python-docx lxml reportlab

# If 'python' command fails, use the Windows launcher:
py -m pip install requests beautifulsoup4 python-docx lxml reportlab

# Install from requirements.txt:
python -m pip install -r requirements.txt
```

### B. Running Course Comparisons (Python Main Script):

```
# Single course comparison:
python main.py --legacy-url "https://learn.adbanker.com/dashboard.aspx" --new-url "https://adbanker-stag
ing.myshuttle.io" --out Audit_Report.docx

# Batch multi-course comparison using parallel worker threads:
python main.py --legacy-url urls_a.txt --new-url urls_b.txt --workers 16 --out Multi_Course_Report.docx

# Run quick built-in diagnostic test:
python main.py --workers 16
```

### C. Zip Extraction on a New Machine:

```
# Extract course-auditor-main-updated.zip to Desktop:
Expand-Archive -Path "$env:USERPROFILE\Desktop\course-auditor-main-updated.zip" -DestinationPath "$env:U
SERPROFILE\Desktop\course-auditor" -Force

# Navigate into folder and install Node.js dependencies:
cd "$env:USERPROFILE\Desktop\course-auditor"
npm install
npm run audit
```

### D. GitHub CLI Setup & Repository Push:

```
# Step 1: Install GitHub CLI via winget:
winget install --id GitHub.cli --accept-package-agreements --accept-source-agreements

# Step 2: One-time browser authentication:
gh auth login --web -h github.com -p https

# Step 3: Initialize, commit, and push repository:
cd "C:\Users\HP OMEN\Downloads\course-auditor-main"
git init
git add .
git commit -m "Update course-auditor with human-judgment rules"
gh repo create course-auditor --public --source=. --push
```

## 5. Key Code Algorithms & False-Positive Fixes

### A. Typography Sanitization (Strip Bold / Italics Before Comparing):

```
function stripTypographyFormatting(text) {
  if (!text) return '';
  return text
    .replace(/<\/?(b|strong|i|em|span)[^>]*>/gi, '') // Strip HTML tags
    .replace(/(\*\*|__|\*|_)/g, '')              // Strip Markdown bold/italics
    .replace(/\s+/g, ' ')                          // Normalize whitespace
    .trim();
}
```

### B. Occurrence-Count Duplicate Heading Verification:

```
const uniqueSlideHeadings = new Set(slideHeadings);
for (const h of uniqueSlideHeadings) {
  const countNew = slideHeadings.filter(x => x === h).length;
  const countLegacy = sourceHeadings.filter(x => x === h).length;
  // Flag ONLY if New System contains an EXTRA occurrence compared with Legacy:
  if (countNew > countLegacy) {
    issues.push({
      differenceType: 'DUPLICATE HEADING',
      explanation: `Heading "${h}" appears ${countNew} times in New vs ${countLegacy} in Legacy.`
    });
  }
}
```

### C. Active HTTP Link Health Validation (Testing 200 vs 404/500):

```
def check_link_health(url: str) -> dict:
    if not url or url.startswith("#") or "javascript:" in url:
        return {"working": True, "status_code": 200, "error": None}
    try:
        resp = session.head(url, timeout=5, allow_redirects=True)
        if resp.status_code >= 400:
            resp = session.get(url, timeout=5, stream=True)
        working = resp.status_code < 400
        return {"working": working, "status_code": resp.status_code, "error": None if working else f"HTT
P {resp.status_code}"}
    except Exception as e:
        return {"working": False, "status_code": 0, "error": f"Connection Failed ({type(e).__name__})"}
```

### D. Visual Bounding-Box Overlap Check (Playwright):

```
async function detectVisualOverlaps(page) {
  const images = await page.$$('img, svg, .image-container');
  const textContainers = await page.$$('.callout, .card, p, table');
  for (const img of images) {
    const imgBox = await img.boundingBox();
    if (!imgBox) continue;
    for (const box of textContainers) {
      const textBox = await box.boundingBox();
      if (!textBox) continue;
      const overlaps = !(
        imgBox.x + imgBox.width <= textBox.x || textBox.x + textBox.width <= imgBox.x ||
        imgBox.y + imgBox.height <= textBox.y || textBox.y + textBox.height <= imgBox.y
      );
      if (overlaps) return { hasOverlap: true, element: 'Image overlapping text container' };
    }
  }
  return { hasOverlap: false };
}
```

## 6. Fast 9-to-5 Operational Strategy (12 Courses)

**Phase 1: Automated Report Cleaner (Takes 1 Minute)**
Run Prompt 2 across bloated 97-error reports to auto-eliminate 70% of noise.

**Phase 2: Pattern Clustering (Takes 5 Minutes per Course)**
Review errors in 4 buckets: Duplicate Titles, Truncated Paragraphs, Deformed Callouts, Broken 404 Links.

**Phase 3: Targeted Spot-Checking**
Check only Slide 1 (title duplicates), Last Slide (truncated text), and Table slides. Trust 100% matches on middle slides.

**Phase 4: Ground-Truth Calibration (Chapters 1–3)**
Manually review Chapters 1–3 of 1 course. Feed results into Prompt 3 to permanently align model criteria across all 12 courses.